INTERNSPARK CYBERSECURITY INTERNSHIP

Vulnerability Assessment Report

Smart Expense Tracker Web Application

Task 1: Reconnaissance & Vulnerability Scanning

Application	Smart Expense Tracker
Authorized target	https://smart-expense-tracker-0pon.onrender.com/login
Author	Raghav Marda
Assessment date	18 August 2026 (from Nmap scan timestamps, IST +05:30)
Primary tool	Nmap 7.99.1 (Windows Command Prompt)
Classification	Academic / internship submission. Authorized self-assessment of the author's own application.

Table of Contents

1. Executive Summary 3

2. Introduction 3

3. Assessment Objectives 4

4. Scope and Authorization 4

5. Target Application Profile 4

6. Tools and Technologies 5

7. Methodology 5

8. Detailed Scan Results 6

9. Detailed Findings 9

10. Automated Checks With No Findings 11

11. Inconclusive Checks and Limitations 12

12. Risk Assessment Summary 12

13. Remediation Plan 13

14. Conclusion 13

15. References and Evidence 14

1. Executive Summary

This report documents an authorized reconnaissance and vulnerability-scanning exercise carried out against *Smart Expense Tracker*, a Python/Flask web application deployed on Render by the author, as Task 1 of the InternSpark Cybersecurity Internship. The purpose was to practise active reconnaissance and automated vulnerability checks on a safe, owned system and to document the results responsibly.

Nmap 7.99.1, run from Windows Command Prompt, was the only tool used. Five activities were performed: a basic TCP port scan, service/version detection on ports 80 and 443, the Nmap `vuln` NSE script category against ports 80 and 443, HTTP title and response-header inspection, and an HTTPS security-header check. A sixth attempt (`http-enum`) failed because of a DNS resolution error and produced no results. Nikto and OpenVAS were **not** used.

Key observations. (1) Nmap did not detect a Strict-Transport-Security (HSTS) header on the HTTPS endpoint. (2) A response header, `x-render-origin-server`, exposed the Werkzeug and Python version strings. (3) Nmap flagged a potential Slowloris exposure (`LIKELY VULNERABLE`) in one scan attempt. (4) Nine ports were reported open on the scanned address, which resolves to Render's ingress infrastructure and is behind Cloudflare.

How to read these results. None of the observations is a confirmed, exploited vulnerability. The Slowloris result is an unconfirmed indicator that needs safe validation. The open ports are an infrastructure observation, not automatically an application flaw. Several checks were inconclusive (a script error, a timeout, and a failed DNS lookup), and the automated XSS and CSRF checks that reported nothing do not prove their absence. The assessment does not establish that the application is secure.

> **Evidence verification note.** Every screenshot was inspected against the written scan record. Screenshots `01`, `02` and `03` match the record. Screenshots `04_http_headers.png` and `05_security_headers.png` instead show a DNS failure ("Failed to resolve", 0 hosts scanned) and do **not** display the header or HSTS output. Likewise, the `http-slowloris-check` line is **not** visible in `03_vulnerability_scan.png`. Findings F-01, F-02 and F-03 therefore rest on the assessor's session notes and are labelled accordingly. Re-running those commands and capturing fresh screenshots is recommended before final submission (see Sections 11 and 13).

2. Introduction

2.1 Reconnaissance

Reconnaissance is the information-gathering phase of a security assessment. *Active* reconnaissance sends probes directly to the target (for example, TCP connection attempts) to learn which hosts, ports and services are reachable, and what software they appear to run. The output gives an assessor a map of the externally visible attack surface.

2.2 Vulnerability scanning

Vulnerability scanning builds on that map by running automated checks that look for known weaknesses, misconfigurations and missing protections. Automated scanners are fast and repeatable but produce false positives, false negatives and inconclusive results. Their output is a set of leads to be interpreted and validated by a person, not a final verdict.

2.3 Purpose of this task

Task 1 required the intern to use Nmap for port and service enumeration, perform vulnerability scanning on a safe test system, capture screenshots, document findings and potential risks, and prepare a short assessment report. This document is that report.

3. Assessment Objectives

Activity	Objective
Port scanning	Identify which TCP ports on the target hostname respond, and in what state (open, closed, filtered).
Service detection	Determine what software answers on the web ports (80 and 443) using Nmap version detection.
Vulnerability checks	Run Nmap's `vuln` NSE scripts to look for known issues and common web weaknesses, and record what each script reports or fails to report.
HTTP header inspection	Capture the HTTP title, redirect behaviour and response headers, and note any technology information they reveal.
Security-header analysis	Check whether the HTTPS endpoint sends recommended browser security headers, starting with HSTS.

4. Scope and Authorization

4.1 Authorized target

Target URL	https://smart-expense-tracker-0pon.onrender.com/login
Hostname scanned	smart-expense-tracker-0pon.onrender.com
Authorization	The application is owned and deployed by the author, who was authorized to assess it.

4.2 Activities performed

- TCP port scan of the hostname (default Nmap top-1000 ports).
- Service/version detection on ports 80 and 443.
- Nmap `vuln` script category on ports 80 and 443.
- HTTP title and header inspection on ports 80 and 443.
- HTTPS security-header inspection on port 443.
- One `http-enum` attempt (failed; no results).

4.3 Activities not performed

- No Nikto scan (the tool could not be installed; WSL was not available) and no OpenVAS scan.
- No penetration test, exploitation, authentication-bypass or credential testing.
- No source-code review or authenticated testing of application functionality.
- No denial-of-service testing of any kind, including no Slowloris attack.
- No testing of the underlying Render or Cloudflare infrastructure beyond passive observation of what the hostname returns.

5. Target Application Profile

Attribute	Detail
Application	Smart Expense Tracker
Language / framework	Python / Flask
Database	SQLite

Attribute	Detail
Hosting	Render
Edge layer	Cloudflare proxy/edge layer observed during scanning (Server: cloudflare; Nmap identified "Cloudflare http proxy")
Resolved addresses	The hostname resolved to two IPs (216.24.57.7 and 216.24.57.15). Reverse DNS for each is of the form ip-216-24-57-N.ingress.render.com.

No further architecture details were supplied or are assumed in this report.

6. Tools and Technologies

Tool / technology	Role in this assessment
Nmap 7.99.1	Primary and only assessment tool: port scan, version detection, NSE scripts.
Nmap NSE scripts	`vuln` category, `http-title`, `http-headers`, `http-security-headers`, `http-enum` (failed).
Windows Command Prompt	Execution environment for all commands.
Python / Flask, SQLite	Technology of the assessed application (context only; not directly tested).
Render, Cloudflare	Hosting platform and proxy/edge layer in front of the target (context only).

Not used: Nikto and OpenVAS. The command `nikto -Version` returned a "not recognized" error, and `wsl --status` confirmed WSL was not installed. No result in this report comes from either tool.

7. Methodology

The assessment moved from broad to specific: discover what is reachable, identify what answers on the web ports, run automated checks, then inspect the HTTP layer. Each stage and the information it was intended to reveal is listed below.

#	Command	Purpose / information sought
1	nmap smart-expense-tracker-0pon.onrender.com	Default TCP scan of the 1000 most common ports. Reveals port states and Nmap's default service labels.
2	nmap -sV -p 80,443 smart-expense-tracker-0pon.onrender.com	Version detection on the web ports. Reveals what software answers behind the labels.
3	nmap --script vuln -p 80,443 smart-expense-tracker-0pon.onrender.com	Runs Nmap's vulnerability-oriented NSE scripts against the web ports to look for known issues and common web weaknesses.
4	nmap --script http-title,http-headers -p 80,443 smart-expense-tracker-0pon.onrender.com	Retrieves page title, redirects and response headers to review redirect behaviour and technology disclosure.
5	nmap --script http-security-headers -p 443 smart-expense-tracker-0pon.onrender.com	Checks the HTTPS response for recommended security headers (HSTS reported).
6	nmap --script http-enum -p 80,443 smart-expense-tracker-0pon.onrender.com	Intended to enumerate common web paths. Failed due to DNS resolution; no output of value.

All scans were run from the author's own Windows machine against the single hostname above. Scans were non-destructive: no exploitation scripts were forced and no denial-of-service technique was run.

8. Detailed Scan Results

All scans ran on 18 August 2026 (IST). Because the hostname resolves to two IP addresses, Nmap chose one of them per run (216.24.57.7 for scans 1 and 2; 216.24.57.15 for scan 3), as stated in each screenshot.

8.1 Basic Port Scan

Command	`nmap smart-expense-tracker-0pon.onrender.com`
Purpose	Identify reachable TCP ports on the target hostname.
Evidence status	**Screenshot-verified** (01_nmap_port_scan.png)

Observed output

```
Starting Nmap 7.991 ( https://nmap.org ) at 2026-08-18 23:23 +0530
Nmap scan report for smart-expense-tracker-0pon.onrender.com (216.24.57.7)
Host is up (0.029s latency).
Other addresses (not scanned): 216.24.57.15
rDNS record for 216.24.57.7: ip-216-24-57-7.ingress.render.com
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE
25/tcp   open   smtp
80/tcp   open   http
110/tcp  open   pop3
113/tcp  closed ident
143/tcp  open   imap
443/tcp  open   https
8010/tcp open   xmpp
8080/tcp open   http-proxy
8443/tcp open   https-alt
Nmap done: 1 IP address (1 host up) scanned in 7.86 seconds
```

Technical interpretation

Nine ports were listed: eight open and one closed (113/ident), with 991 filtered. The service names are Nmap's default labels based on port number, not confirmed identification; the presence of, for example, 25, 110 and 143 does not show that mail services are running. The reverse-DNS name points to Render's ingress layer, and the site is fronted by Cloudflare, so these ports most likely reflect shared hosting/proxy infrastructure rather than the Flask application. This was not verified.

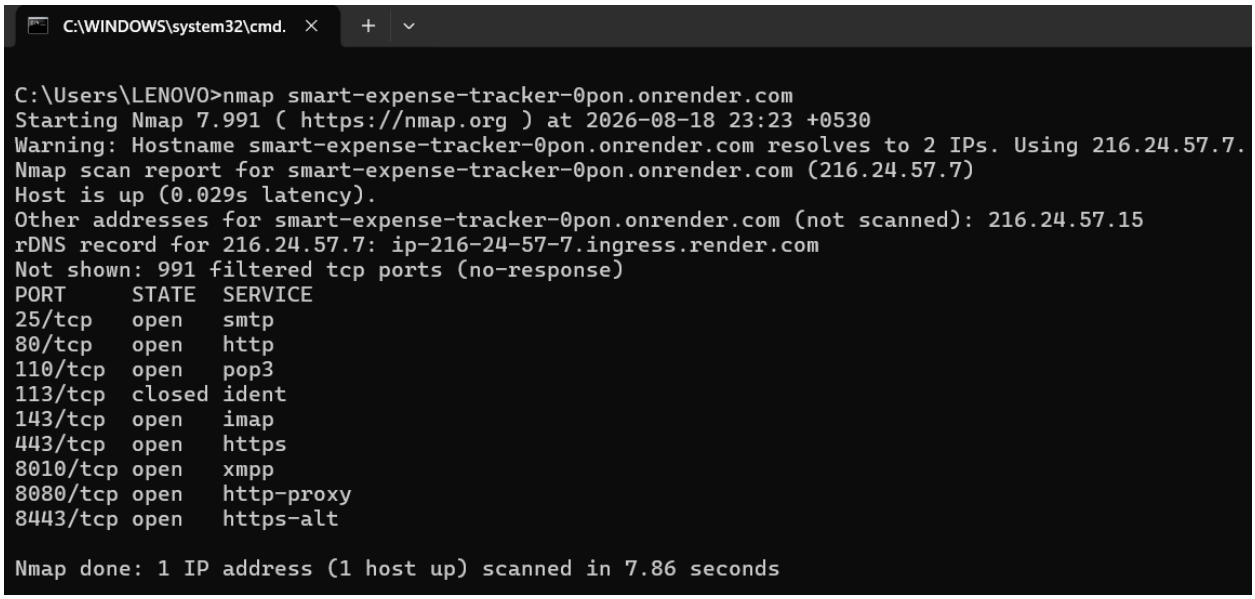


Figure 1: 01_nmap_port_scan.png - basic Nmap port scan.

8.2 Service and Version Detection

Command	`nmap -sV -p 80,443 smart-expense-tracker-0pon.onrender.com`
Purpose	Identify the software responding on the HTTP and HTTPS ports.
Evidence status	**Screenshot-verified** (02_service_version_detection.png)

Observed output

```
Nmap scan report for smart-expense-tracker-0pon.onrender.com (216.24.57.7)
Host is up (0.0083s latency).
PORT    STATE SERVICE  VERSION
80/tcp  open  http     Cloudflare http proxy
443/tcp open  ssl/http Cloudflare http proxy
Nmap done: 1 IP address (1 host up) scanned in 15.50 seconds
```

Technical interpretation

Nmap identified the Cloudflare proxy on both ports. This confirms that the web ports answer through a Cloudflare edge layer, so the scan describes that layer and does not establish the configuration of the Flask application behind it.

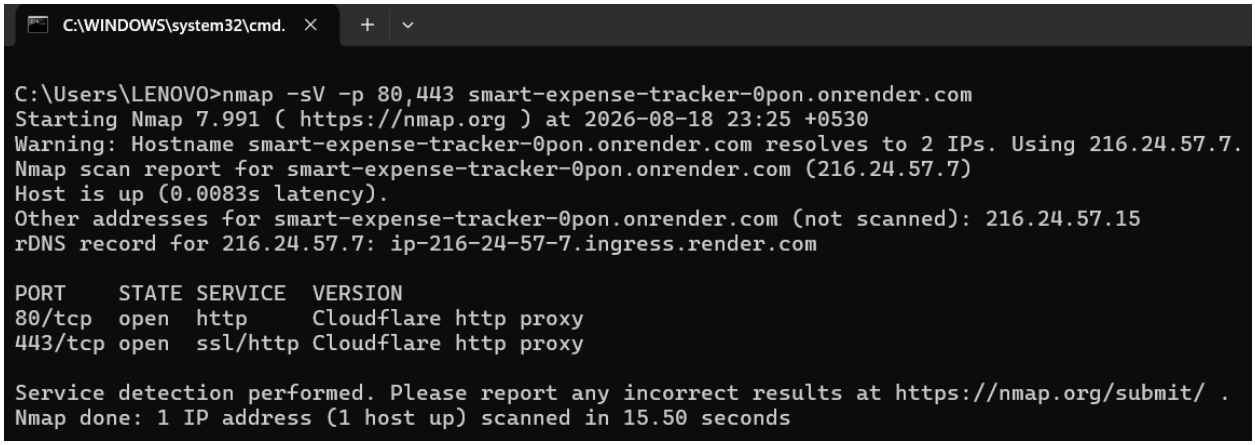


Figure 2: 02_service_version_detection.png - service/version detection.

8.3 Nmap Vulnerability Script Scan

Command	`nmap --script vuln -p 80,443 smart-expense-tracker-0pon.onrender.com`
Purpose	Run Nmap's vulnerability-oriented NSE scripts against the web ports.
Evidence status	**Screenshot-verified** for the lines below (03_vulnerability_scan.png). The Slowloris line is **Session notes only** (see F-03).

Observed output (transcribed from screenshot)

```
Starting Nmap 7.991 ( https://nmap.org ) at 2026-08-18 23:26 +0530
Warning: Hostname ... resolves to 2 IPs. Using 216.24.57.15.
Host is up (0.0036s latency).
rDNS record for 216.24.57.15: ip-216-24-57-15.ingress.render.com
PORT    STATE SERVICE
80/tcp  open  http
|_http-vuln-cve2014-3704: ERROR: Script execution failed (use -d to debug)
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
443/tcp open  https
|_ssl-ccs-injection: No reply from server (TIMEOUT)
| http-fileupload-exploiter:
|_    Couldn't find a file-type field.
```

```
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-csrf: Couldn't find any CSRF vulnerabilities.
Nmap done: 1 IP address (1 host up) scanned in 1020.06 seconds
```

Technical interpretation

- **No findings reported:** DOM-based XSS, stored XSS and CSRF checks (both ports) and the file-upload check (it found no file-type field to test). These are covered in Section 10.
- **Inconclusive:** `http-vuln-cve2014-3704` failed to execute, and `ssl-ccs-injection` timed out. Neither is evidence of vulnerability (Section 11).
- **Slowloris:** the written scan record states one attempt reported `http-slowloris-check: State: LIKELY VULNERABLE`. That line does not appear in the supplied screenshot; see F-03.
- The scan took about 17 minutes (1020.06 s). The assessor's notes state that results differed somewhat between attempts, which is why a single run should not be treated as definitive.

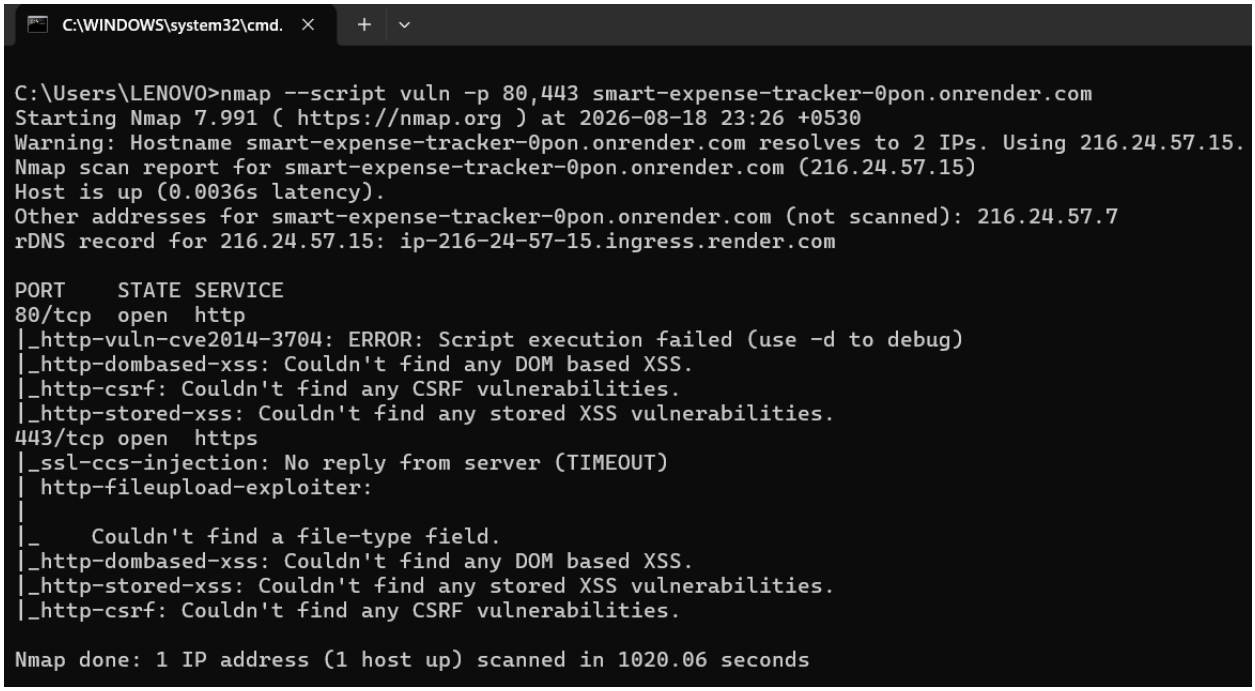


Figure 3: 03_vulnerability_scan.png - Nmap vuln script scan.

8.4 HTTP Title and Response Headers

Command	`nmap --script http-title,http-headers -p 80,443 smart-expense-tracker-0pon.onrender.com`
Purpose	Retrieve redirect behaviour, page title and response headers.
Evidence status	**Session notes only**. The supplied screenshot (04_http_headers.png) shows a DNS failure, not the results below.

Observed results (from assessor's session notes)

- HTTP (port 80) redirected to HTTPS: `Location: https://smart-expense-tracker-0pon.onrender.com/`
- The HTTPS response redirected toward `/login`.
- Headers included: `Server: cloudflare`; `x-render-origin-server: Werkzeug/3.1.8 Python/3.14.3`; `cf-cache-status: DYNAMIC`; `alt-svc: h3=":443"; ma=86400`.

Technical interpretation

The HTTP-to-HTTPS redirect is the expected behaviour for an HTTPS-only application. The `Server` and `cf-cache-status` headers are consistent with the Cloudflare edge layer seen in scan 8.2. The `x-render-origin-server` header discloses technology and version strings and is discussed as F-02.

Screenshot as supplied

The screenshot for this command (timestamp 23:44) shows Nmap failing to resolve the hostname and scanning 0 hosts. It is included below for transparency and should not be read as evidence of the header output.

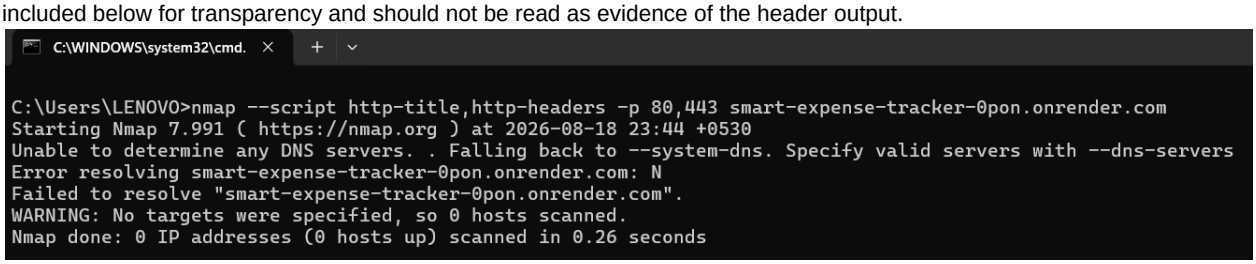


Figure 4: 04_http_headers.png - supplied screenshot shows a DNS resolution failure (0 hosts scanned).

8.5 HTTP Security Headers

Command	`nmap --script http-security-headers -p 443 smart-expense-tracker-0pon.onrender.com`
Purpose	Check the HTTPS response for recommended browser security headers.
Evidence status	**Session notes only**. The supplied screenshot (05_security_headers.png) shows a DNS failure, not the result below.

Observed result (from assessor's session notes)

```
Strict_Transport_Security: HSTS not configured in HTTPS Server
```

Technical interpretation

Nmap did not detect a Strict-Transport-Security header at the scanned HTTPS endpoint. The notes record only this line, so no statement is made here about any other security header (for example CSP, X-Frame-Options or X-Content-Type-Options); they are neither reported as missing nor as present.

Screenshot as supplied

The screenshot for this command (timestamp 23:44) shows the same DNS failure as above.

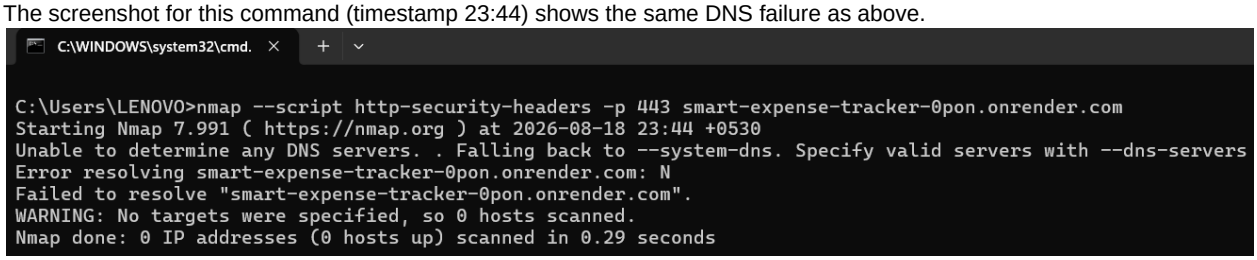


Figure 5: 05_security_headers.png - supplied screenshot shows a DNS resolution failure (0 hosts scanned).

9. Detailed Findings

Findings are separated into observations and potential issues. No CVSS scores are assigned, because they cannot be justified from scan output alone. Severity labels are the assessor's qualitative judgement and are given only where the evidence supports one.

F-01: HSTS Header Not Detected

Finding ID	F-01
Category	Security-header configuration (observed)
Evidence	Nmap `http-security-headers` output: "HSTS not configured in HTTPS Server" (Section 8.5). Evidence status: **Session notes only**; the supplied screenshot 05 does not show it.
Description	The scanned HTTPS endpoint did not return a Strict-Transport-Security header according to Nmap.
Potential impact	Without HSTS, a browser is not told by this header to insist on HTTPS for future visits. In some situations (for example, a user on a hostile network typing the bare hostname or following an http:// link) this can leave a window for downgrade or SSL-stripping attacks. The application already redirects HTTP to HTTPS, which reduces but does not remove this window.
Recommended remediation	Once the application is confirmed to be HTTPS-only, send a Strict-Transport-Security header, starting with a short max-age and increasing it after testing. Because the site is served on an onrender.com hostname, edge settings are likely managed by the platform, so setting the header in the Flask application's responses is the practical route. Avoid includeSubDomains and preload unless they are clearly appropriate.
Confidence / limitations	Moderate. The check is simple and the result is plausible, but it must be re-run with a captured screenshot. A proxy or hosting layer could affect which headers reach the client.
Severity	Low (qualitative). Further validation required; no exploitability was tested.

F-02: Technology/Version Information in Response Header

Finding ID	F-02
Category	Potential information disclosure
Evidence	Header `x-render-origin-server: Werkzeug/3.1.8 Python/3.14.3` (Section 8.4). Evidence status: **Session notes only**; the supplied screenshot 04 does not show it.
Description	A response header revealed the Werkzeug and Python version strings. The header name suggests it is added in the Render hosting path, but its exact origin was not verified.
Potential impact	Version strings can help an attacker choose targeted techniques. This report does **not** claim the disclosed versions are vulnerable; no supporting evidence for that was gathered.
Recommended remediation	Check whether the header is generated by the platform or the application and whether it can be removed or reduced. Also confirm which WSGI server runs in production: the string format resembles what Werkzeug's built-in development server emits, and the built-in server is not intended for production use. This was not verified.
Confidence / limitations	Moderate that the header was present (per notes); low on its origin and significance.
Severity	Informational to Low (qualitative). Further validation required.

F-03: Potential Slowloris Exposure Flagged by Nmap

Finding ID	F-03
Category	Unconfirmed potential exposure
Evidence	The scan record states one scan output contained `http-slowloris-check: State: LIKELY VULNERABLE`. Evidence status: **Session notes only**. The line is **not** visible in the supplied 03_vulnerability_scan.png, whose output lists the other scripts shown in Section 8.3.
Description	Nmap's check indicated the web server might be susceptible to slow-connection resource exhaustion. The script checks for susceptibility and does not launch a denial-of-service attack.
Potential impact	If genuine, slow-connection attacks could tie up server connections and degrade availability. The target sits behind Cloudflare and Render's ingress layer, which commonly absorb or terminate such connections, so the result may not reflect real exposure of the Flask application.
Recommended remediation	Re-run only `http-slowloris-check`, capture the output, and compare across attempts. Review proxy/hosting timeout behaviour and the production server's timeout settings. Do not run a Slowloris or any DoS test.
Confidence / limitations	Low. The result appeared in only one attempt, is not in the supplied screenshot, and is a heuristic check on a proxied target.
Severity	Not rated. Further validation is required before any rating can be justified.

F-04: Multiple Open Ports Observed at the Target Hostname

Finding ID	F-04
Category	Network/hosting observation
Evidence	Basic port scan (Section 8.1). Evidence status: **Screenshot-verified** (01_nmap_port_scan.png).
Description	Eight ports were open (25, 80, 110, 143, 443, 8010, 8080, 8443) and one closed (113) on 216.24.57.7. Service names are Nmap's default labels.
Potential impact	Unneeded public services enlarge the attack surface. Here, the IP's reverse DNS points to Render's ingress layer and the site is behind Cloudflare, so the ports are not automatically attributable to the Flask application or under the author's control.
Recommended remediation	Confirm with Render's documentation or support which ports are part of the platform's ingress and which, if any, belong to the application. Ensure the application itself listens only on the intended port. No application-side change is indicated by this evidence alone.
Confidence / limitations	High that the ports responded as listed in that run; low on ownership and purpose. Only one of the two resolved IPs was scanned.
Severity	Informational. Not an application vulnerability on current evidence.

10. Automated Checks With No Findings

Check (NSE script)	Output	Ports
http-csrf	Couldn't find any CSRF vulnerabilities.	80, 443
http-dombased-xss	Couldn't find any DOM based XSS.	80, 443

Check (NSE script)	Output	Ports
http-stored-xss	Couldn't find any stored XSS vulnerabilities.	80, 443
http-fileupload-exploiter	Couldn't find a file-type field.	443

These results mean only that these specific scripts, run against the pages they were able to crawl, did not report an issue. These scripts depend on what the crawler can reach; the application is login-gated, and the scan was unauthenticated, so pages behind login were most likely not examined. The results do not show that the application is free of CSRF or XSS vulnerabilities, and they should not be cited as such. The file-upload result means no upload form was found by the crawler, not that the application has no upload functionality.

11. Inconclusive Checks and Limitations

11.1 Inconclusive checks

Item	What happened	Interpretation
http-vuln-cve2014-3704	ERROR: Script execution failed (use -d to debug)	The script did not complete. This is not evidence of vulnerability. Re-running with -d would show why.
ssl-ccs-injection	No reply from server (TIMEOUT)	No conclusive result. A timeout is not a vulnerability.
http-enum (06_http_enumeration.png)	Unable to determine any DNS servers; Failed to resolve; 0 hosts scanned	Local DNS failure. No directories or resources were enumerated, and none are reported.
http-headers / http-security-headers screenshots (04, 05)	Same DNS failure at 23:44	The screenshots do not show the intended output (see Section 8.4 and 8.5).

11.2 Limitations

- **Proxy/hosting layer:** the target is behind Cloudflare and Render's ingress. Port, service and header results may describe those layers rather than the Flask application.
- **Two IP addresses:** the hostname resolved to two addresses; each scan used one, and not always the same one.
- **Single-run results:** the notes record that results differed between attempts, so single runs are not definitive.
- **Tools:** only Nmap was used. Nikto and OpenVAS were not, so web-server misconfiguration and broader vulnerability coverage from those tools is absent.
- **Scope of automation:** unauthenticated, automated scanning only. No manual testing, authenticated testing, business-logic review, injection testing, session testing or source-code audit was performed.
- **Evidence gaps:** F-01, F-02 and F-03 are not supported by the supplied screenshots.

12. Risk Assessment Summary

Category	Item	Status	Evidence	Indicative rating
Observed configuration issue	F-01 HSTS not detected	Observed (to be re-verified)	Session notes	Low

Category	Item	Status	Evidence	Indicative rating
Observed configuration issue	F-02 Version info in header	Observed (to be re-verified)	Session notes	Info / Low
Potential exposure requiring validation	F-03 Slowloris 'LIKELY VULNERABLE'	Unconfirmed	Session notes	Not rated
Infrastructure observation	F-04 Open ports 25, 80, 110, 143, 443, 8010, 8080, 8443	Observed; ownership unverified	Screenshot 01	Informational
Inconclusive	CVE-2014-3704 script error	Inconclusive	Screenshot 03	N/A
Inconclusive	SSL CCS injection timeout	Inconclusive	Screenshot 03	N/A
Inconclusive	http-enum DNS failure	Failed attempt	Screenshot 06	N/A
No finding reported	CSRF, DOM XSS, stored XSS, file upload	No finding from automated check	Screenshot 03	N/A (not proof of absence)

Ratings are qualitative and indicative only. None of the items was confirmed as exploitable.

13. Remediation Plan

#	Action	Related	Priority
1	Re-run the header and HSTS scans and capture screenshots of the actual output.	F-01, F-02	Do first
2	Review and configure HSTS appropriately (start with a short max-age; HTTPS-only confirmed first).	F-01	Medium
3	Check whether the version-bearing header can be removed or reduced; confirm the production WSGI server.	F-02	Low
4	Re-run only `http-slowloris-check` several times, capture output, and review proxy/server timeouts. No DoS testing.	F-03	Medium
5	Verify with Render which open ports are platform-owned; ensure only intended services are exposed by the application.	F-04	Low
6	Plan further authorized testing: authenticated scanning, Nikto/OpenVAS (or equivalent) once available, and manual review of login, session and input handling.	All	Follow-up
7	Repeat relevant scans after changes and compare against this baseline.	All	After fixes

14. Conclusion

The assessment demonstrated basic reconnaissance and service enumeration with Nmap, automated NSE vulnerability checks, HTTP header inspection and security-header review on an authorized, self-owned deployment. The work gave practical experience in interpreting scanner output critically, including recognising that a proxied target, a script error or a timeout does not map directly to a vulnerability.

Key results: Nmap did not detect an HSTS header; a response header exposed Werkzeug and Python version strings; Nmap flagged a potential, unconfirmed Slowloris exposure in one attempt; and nine ports were reported on an address belonging to Render's ingress layer. Several checks were inconclusive, and the automated XSS and CSRF checks reported nothing, which is not proof of absence. The application cannot be called secure or insecure on this evidence.

Remaining work: re-run the header, HSTS and Slowloris checks with captured screenshots; confirm port ownership with the hosting provider; apply proportionate hardening; and carry out deeper authorized testing, including authenticated and manual testing, followed by re-scanning.

15. References and Evidence

15.1 References

- Nmap Reference Guide (official manual): https://nmap.org/book/man.html
- Nmap Scripting Engine script documentation: https://nmap.org/nsedoc/
- RFC 6797, HTTP Strict Transport Security (HSTS): https://www.rfc-editor.org/rfc/rfc6797

Nmap is the primary assessment tool. References are standard official documentation and were not used to generate any result.

15.2 Screenshot evidence index

File	Command	What the screenshot actually shows	Supports
01_nmap_port_scan.png	nmap [target]	Successful port scan; 8 open, 1 closed, 991 filtered	F-04, Sec. 8.1
02_service_version_detection.png	nmap -sV -p 80,443	Cloudflare http proxy on 80 and 443	Sec. 8.2
03_vulnerability_scan.png	nmap --script vuln -p 80,443	Script results listed in Sec. 8.3; no Slowloris line visible	Sec. 8.3, 10, 11
04_http_headers.png	http-title,http-headers	DNS failure; 0 hosts scanned. Headers not shown	None (see F-02 note)
05_security_headers.png	http-security-headers	DNS failure; 0 hosts scanned. HSTS result not shown	None (see F-01 note)
06_http_enumeration.png	http-enum	DNS failure; 0 hosts scanned	Failed attempt (Sec. 11)

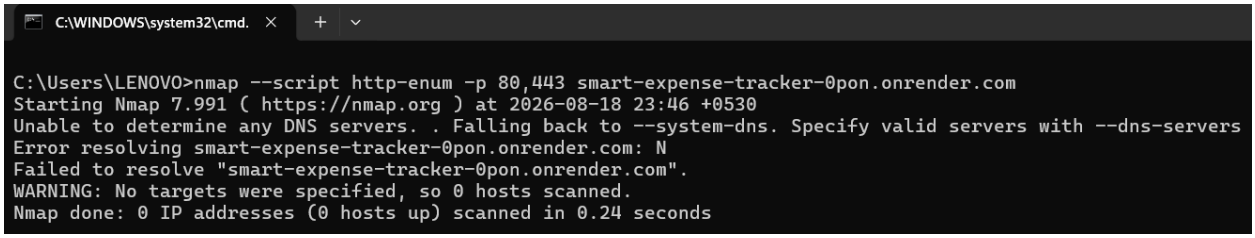


Figure 6: 06_http_enumeration.png - failed http-enum attempt (not scan evidence).